

Getting started with Amazon DynamoDB

AWS Prescriptive Guidance

Amazon DynamoDB is a fully managed, serverless, key-value and document NoSQL database that delivers single-digit millisecond performance at any scale. This guide covers DynamoDB data modeling, partition and sort keys, secondary indexes, capacity modes, DynamoDB Streams, global tables, transactions, DAX caching, and best practices for designing highly scalable and cost-efficient DynamoDB applications.

Authors: Lena Müller and Arjun Kapoor, Amazon Web Services (AWS)

Date: March 2026

Table of Contents

- [Introduction](#)
- [Core Concepts](#)
- [Data Modeling](#)
- [Secondary Indexes](#)
- [Capacity Modes](#)
- [DynamoDB Streams](#)
- [Global Tables](#)
- [Transactions](#)
- [DynamoDB Accelerator \(DAX\)](#)
- [Security](#)
- [Best Practices](#)
- [FAQ](#)

Introduction

Amazon DynamoDB is a key-value and document database designed for applications that need consistent, single-digit millisecond latency at any scale. It is a fully managed cloud service with built-in security, backup and restore, and in-memory caching. DynamoDB can handle more than 10 trillion requests per day and can support peaks of more than 20 million requests per second.

DynamoDB was designed around the principles of Werner Vogels's Dynamo paper and offers automatic multi-AZ replication, automatic sharding, and automatic storage scaling. You do not need to manage cluster provisioning, hardware, configuration, replication, software patching, or cluster scaling. DynamoDB is ideal for web, mobile, gaming, ad tech, IoT, and many other applications.

Note: DynamoDB automatically spreads the data and traffic for your tables over a sufficient number of servers to handle your throughput and storage requirements, while maintaining consistent and fast performance.

Core Concepts

Tables, Items, and Attributes

A DynamoDB table is a collection of data. Each table contains zero or more items. An item is a collection of attributes — similar to a row in a relational database — but unlike relational databases, items in the same table can have different attributes. An attribute is a fundamental data element. DynamoDB supports scalar types (String, Number, Binary, Boolean, Null), document types (List, Map), and set types (String Set, Number Set, Binary Set).

Primary Key

Every DynamoDB table must have a primary key, which uniquely identifies each item. DynamoDB supports two types of primary keys. A simple primary key uses only a partition key (hash key). A composite primary key uses a partition key and a sort key (range key). The combination of partition key and sort key must be unique across all items. The partition key value is input to an internal hash function that determines which physical partition stores the item.

Data Modeling

Data modeling in DynamoDB is fundamentally different from relational database design. Because DynamoDB is optimized for performance at scale, your data model must be designed around the access patterns your application requires — not around normalization or entity relationships.

The single-table design pattern is a popular DynamoDB data modeling approach where multiple entity types are stored in the same table. This pattern eliminates the need for joins (which DynamoDB does not support) by pre-joining related data. Overloading the partition key and sort key with composite values (such as `USER#userId` and `ORDER#orderId`) allows different entity types to coexist and be queried efficiently.

The key principle of DynamoDB data modeling is: write your access patterns first, then design the table structure and indexes to support them efficiently. Changing your access patterns after the table is in production is more complex than in relational databases.

Secondary Indexes

Index Type	Key Schema	Projection	Consistency	When to Use
GSI (Global Secondary Index)	Partition key + optional sort key (any attributes)	Some, all, or keys only	Eventually consistent	Access by non-primary key attributes
LSI (Local Secondary Index)	Same partition key, different sort key	Some, all, or keys only	Strong or eventual	Alternate sort orders on same partition

Global Secondary Indexes (GSIs) have their own provisioned throughput independent of the base table. You can add GSIs to an existing table at any time, whereas LSIs must be defined at table creation time. DynamoDB supports up to 20 GSIs and 5 LSIs per table.

Capacity Modes

DynamoDB offers two capacity modes. On-Demand capacity mode automatically scales read and write throughput up and down based on your application's traffic without capacity planning. You pay per request for the data reads and writes your application performs. On-Demand is ideal for new tables with unknown workloads or unpredictable traffic.

Provisioned capacity mode allows you to specify the number of reads and writes per second your application requires. You can use auto scaling to automatically adjust your table's capacity in response to changes in traffic. Provisioned mode with consistent traffic patterns and reserved capacity (Committed Throughput) can significantly reduce costs.

DynamoDB read and write operations are measured in Read Capacity Units (RCUs) and Write Capacity Units (WCUs). One RCU provides one strongly consistent read per second, or two eventually consistent reads per second, for an item up to 4 KB in size. One WCU provides one write per second for an item up to 1 KB in size.

DynamoDB Streams

DynamoDB Streams captures a time-ordered sequence of item-level modifications in a DynamoDB table and stores this information in a log for up to 24 hours. Applications can access this log and view the data items as they appeared before and after they were modified, in near-real time. DynamoDB Streams integrates with AWS Lambda to trigger functions when items are created, updated, or deleted, enabling event-driven architectures.

Global Tables

DynamoDB Global Tables is a fully managed, serverless, multi-Region, and multi-active database. Global Tables automatically replicates your DynamoDB tables across your choice of AWS Regions, providing 99.999% availability SLA and enabling disaster recovery with an RPO and RTO of zero for regional failures. All replica tables in a Global Table accept reads and writes, enabling low-latency local access for globally distributed applications.

Transactions

DynamoDB Transactions provide atomic, consistent, isolated, and durable (ACID) operations on multiple items within and across DynamoDB tables. You can use `TransactWriteItems` to make coordinated all-or-nothing changes to multiple items. `TransactGetItems` enables you to atomically retrieve multiple items from one or more tables. Transactions are ideal for financial ledgers, order management, and any workflow requiring multi-item consistency guarantees.

DynamoDB Accelerator (DAX)

Amazon DynamoDB Accelerator (DAX) is a fully managed, highly available, in-memory cache for DynamoDB that delivers up to a 10x performance improvement — from milliseconds to microseconds — for read-heavy workloads, without requiring application code changes. DAX handles the heavy lifting of adding in-memory acceleration to your DynamoDB tables, without requiring you to manage cache invalidation, data population, or cluster management.

Best Practices

- Design for access patterns first: Document all application access patterns before designing your table schema. Let access patterns drive your key design and index strategy.
- Use single-table design for related entities: Consolidate multiple entity types into a single table to eliminate join operations and improve read efficiency.

- **Avoid hot partitions:** Distribute traffic evenly across partitions using high-cardinality partition keys. Avoid using dates, status fields, or boolean values as partition keys.
- **Use sparse indexes:** Create GSIs on attributes that are present in only a subset of items to reduce index size and write costs.
- **Enable TTL for expiring data:** Use Time to Live (TTL) to automatically delete expired items without consuming write capacity, reducing storage costs.
- **Use condition expressions:** Use ConditionExpression in write operations to implement optimistic locking and prevent concurrent update conflicts.
- **Monitor with CloudWatch:** Track ConsumedReadCapacityUnits, ConsumedWriteCapacityUnits, ThrottledRequests, and SuccessfulRequestLatency to detect and resolve issues.
- **Consider DAX for read-heavy workloads:** If your application has read-heavy workloads with repeated reads on the same items, DAX can dramatically improve performance and reduce costs.

Frequently Asked Questions

Q: When should I use DynamoDB instead of a relational database?

A: Use DynamoDB when your application requires consistent, low-latency performance at any scale, has well-defined and relatively simple access patterns, and does not require complex ad hoc queries or joins. Use relational databases for applications with complex relationships, reporting requirements, and flexible querying needs.

Q: How does DynamoDB handle hot partitions?

A: DynamoDB has a built-in mechanism called Adaptive Capacity that automatically redistributes capacity to handle uneven traffic patterns across partitions. For sustained hot partitions, redesign your key schema to distribute traffic more evenly.